1. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки.
   Вы обратили внимание на таблицу с информацией о зарегистрировавшихся покупателях (клиентах).

Таблица имеет следующую схему:

clients (
client_id number, --уникальный id клиента
client_name varchar(255), --имя клиента
client_surname varchar(255), --фамилия клиента
login varchar(30), --логин, который придумал клиент
city_id number, --id города, который указал клиент (в интерфейсе выбирается название города, а в таблицу сохраняется id)
age number, --возраст клиента
reg_date date –дата регистрации на сайте
);

Считаете ли вы данный набор и смысл полей корректным? Если нет, то напишите что по вашему мнению некорректно и какие изменения внесли бы.

Ответ:

Набор полей не полностью корректен.

1. age нужно заменить на дату рождения, потому что возраст меняется и устаревает.
2. login должен быть уникальным и, скорее всего, иметь большую длину или заменить его на email.
3. Не хватает email/phone для связи с клиентом.
4. Дату регистрации лучше хранить с временем, а не только день.
5. Нужно добавить базовые ограничения: PK/UNIQUE/FK, чтобы данные не развалились.

```
create table if not exists cities (
city_id   bigint generated always as identity primary key,
city_name varchar(255) not null unique
);

create table if not exists clients (
client_id      bigint generated always as identity primary key,
client_name    varchar(255) not null,
client_surname varchar(255) not null,
login          varchar(100) not null unique,
city_id        bigint references cities(city_id),
birth_date     date,
reg_date       timestamp not null default now()  --на случай отсутствия даты регистрации
);
```

2. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки.
   Вы обратили внимание на таблицу с информацией о товарах магазина.

Таблица имеет следующую схему:

```
items (
        item_id        number, --уникальный id товара
        item_name      varchar(255), --наименование товара
        item_cost      number –стоимость товара
);
```

Глядя на эту таблицу, вы вспоминаете, что мы живём в рыночной экономике и цена, если не каждый день, то каждый месяц может меняться. К примеру, сезонные повышения цен или наоборот скидки (перед началом учебного года или новым годом). Так же вы знаете что у магазина есть отдел финансовой отчётности, в котором сотрудникам нужно строить различны отчёты (о доходах например) за разные промежутки времени, в том числе и за прошлые месяцы и даже прошлые года.

Как бы вы доработали архитектуру таблицы, чтобы обеспечить историческое хранение стоимости товара?

Ответ : Схема items в целом корректна для базовой витрины (id + название + цена), но для интернет-магазина, если нужна финансовая отчетность за прошлые периоды, то витрину нужно доработать

Минусы:

- item_cost хранит только текущую цену. А цены меняются (скидки, сезонность и т.д.). Если просто обновлять это поле, мы потеряем историю, и отчеты за прошлый месяц/год будут считаться уже по новой цене → это будет неправильно.

- Не указаны важные ограничения: PK на item_id, и логично сделать NOT NULL для названия и цены (чтобы не было пустых товаров).

- Для цены лучше использовать точный тип (условно “денежный”), а не абстрактный number без масштаба.

Как бы я доработала:

1. Оставила бы items как таблицу “паспорт товара”: item_id, item_name (и остальные постоянные атрибуты).

2. Цена — в отдельную таблицу истории цен (например, item_prices):

3. item_id (ссылка на товар)

4. price

5. период действия цены: valid_from и valid_to (у текущей цены valid_to = NULL)

6. При изменении цены: вставляем новую запись в историю, а предыдущей цене закрываем период.

Так сохраняется история стоимости, и можно будет корректно считать доходы за любой период по цене, которая действовала на дату продажи.

sql

```
  -- Основная таблица товаров (без исторических цен)

-- Товары (паспорт товара)

 create table if not exists items (

  item_id    bigint generated always as identity primary key,

  item_name  varchar(255) not null

);


-- История цен

create table if not exists item_prices (

  price_id    bigint generated always as identity primary key,

  item_id     bigint not null references items(item_id),

  price       numeric(10,2) not null,

  valid_from  timestam not null default now(),

  valid_to    timestamp

);


-- Индексы под частые запросы

create index if not exists ix_item_prices_item_id on item_prices(item_id);

create index if not exists ix_item_prices_valid_from on item_prices(valid_from);
```

3. Требуется проверить запрос на корректность и исправить там ошибки, если они есть. Существует учебная схема HR, содержащая таблицы: employees, departments и locations Необходимо получить все отделы0, расположенные в Сеуле в которых все сотрудники, не имеющие менеджера, зарабатывают в общей сложности более 100000. Для решения задачи был написан следующий запрос:

```
SELECT DEPARTMENT_ID,
      DEPARTMENT_NAME,
      SUM(SALARY) TOTAL_SALARY
  FROM EMPLOYEE E,
      DEPARTMENTS D
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID
   AND MANAGER_ID = NULL
   AND LOCATION_ID = (SELECT LOCATION_ID
                      FROM LOCATIONS
                     WHERE CITY = 'SEOUL')
   AND SUM(SALARY) >= 100000
GROUP BY DEPARTMENT_NAME
```

Ответ

```
SELECT d.department_id,
          d.department_name,
         SUM(e.salary) AS total_salary
FROM employees e
 JOIN departments d ON d.department_id = e.department_id
JOIN locations l ON l.location_id = d.location_id
WHERE e.manager_id IS NULL AND l.city = 'Seoul'
GROUP BY d.department_id, d.department_name
HAVING SUM(e.salary) > 100000;
 -- второй вариант, если у нас нет в таблице департамента id для связи с location
SELECT d.department_id, d.department_name
, SUM(e.salary) AS total_salary
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.manager_id IS NULL
AND d.location_id IN ( SELECT location_id FROM locations WHERE city = 'SEOUL' )
GROUP BY d.department_id, d.department_name
HAVING SUM(e.salary) > 100000;
```

---

4. В некоторой комнате на пол уронили карандаш. Объясните почему вы не можете через него перепрыгнуть?

Ответ: Карандаш лежит в углу комнаты вплотную